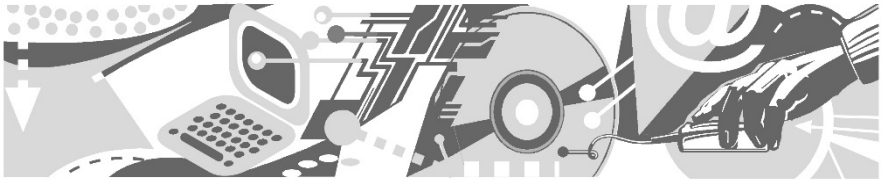


Association Relationships; & Problem Solving

Week

5

LEARNING OBJECTIVES

On completion of this week's learning and activities you should be able to:

- Apply problem solving strategies to methodically break-down a stated problem into smaller aspects for development of a solution;
- Consider the necessary inputs, outputs and processing tasks required to be solved by a program;
- Convert an analysed problem into a functioning object-oriented solution program;
- Model an OO system's design using class diagrams;
- Explain and implement navigability and multiplicity in regards to associations between classes;
- Explain and implement various association relationships including aggregation, composition and temporary relationships, and establishing links between objects; and
- Properly manage the access to items within an aggregate.
- Use the ArrayList class
- Use the For-Each construct to cycle over all elements of an array or an ArrayList
- Implement Data Management classes to manage a set of objects

Contents

1. Introduction 124

2. Problem solving to develop programs 124

3. Representing an analysis of a program using UML 128

4. Association and Aggregation 130

5. Coding Aggregation Relationships in Java 131

6. ArrayList class 134

7. Data Management Classes 138

8. Summary 139

Key Terms 139

Reading

Prescribed readings

The readings that relate to this module come from the following chapters:

Parsons Chapter 5 (particularly section 5.3, but also 5.2)
Chapter 7, "Objects Working Together: Association and Aggregation and Composition"

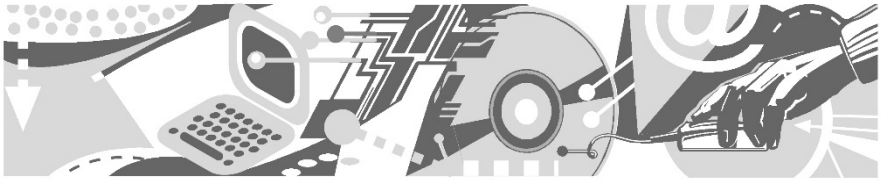
© Federation University Australia

Time allocation

Broadly, the time required to complete this module is shown in the following table, but note this is just a rough indication. You may need to spend more or less time on some activities depending on your background and knowledge.

Working through these session notes	2 hours
Reading Textbook and the lecture slides	2 hours
Attending lab classes/ Doing the lab tasks	2 hours
Other self-directed Study and Review	1 hour
Assignment work	3 hours
Electronic contact (email and discussion forum)	0.5 hours
Total	**10.5 hours**

© Federation University Australia



1. Introduction

This week we begin with looking at how we can approach the task of taking a described problem scenario, to work through an analysis of what is required to designing and eventually writing a computer program that meets the needs of a client/user, by breaking it down into smaller tasks and by representing planned designs graphically.

Many such problems of moderate to large complexity involve multiple domain classes, which may interact with one another by establishing relationships or links between objects. Programs are essentially elaborate scripts of interaction between different types of objects.

In many cases we find that some objects act as owners of other objects, in the sense of having a group of objects of some other type that they take responsibility for managing. We look at how we can implement such grouping by either arrays, or a new class we introduce called the ArrayList.

Finally we look at a rather specific type of grouping relationship, which resembles a database of objects of the one kind, by examining the concept of data management classes.

2. Problem solving to develop programs

The main reason computer programs are written, is to solve some sort of problem from the real-world. This could be a requirement for performing processing of large amounts of data quicker or more frequently than a human could, or for managing information to store for later retrieval or analysis. These are not the only types of problems that a computer program might be desired, but are some of the more common.

In particular, software solution developers are often approached by people or business that have the particular issue they want solved by computer technology, but not knowing the intricacies of programming may only present their needs at a very high level. For example, “We want a new website”, or “we want an app for our customers to be able to buy things from their phone”. Much work is often required to tease-out from the *client*, (the person or business desiring the computer technology solution), exactly what their needs are, and in particularly, what they expect the software to do.

© Federation University Australia

Other units in the degree will cover different aspects of the process, for example, in ITECH2305 *Analysing the Modern Business* (which is not core) you will learn about how to identify in detail what the client's needs are, and in ITECH2002 *Systems Modelling* (which is a core unit), you will learn about ways to analyse problems and design solutions and develop models of how the system needs to be built. This unit provides some of the techniques that may be applicable to the actual building of those designs to meet the identified needs, as do other units with programming as their focus such as ITECH3108 *Dynamic Web Development*.

There is always some degree of *problem solving* required by the programmer once the task to be implemented reaches anything more than a very basic toy task, to be able to take an identified need and transform this into working code. In the following pages we will outline *how* you might step through such a process for an example scenario. We then explore some of the programming techniques that would be needed to be able to implement a solution.

2.1. The Bookshop Scenario

Consider that the bookshop in your local town/suburb, or even the bookshop on your campus of the university, has come to you wanting you to provide them with a new computerised system to help them running their bookshop.

Firstly, it is important to have a bit of a sense of the way the bookshop conducts its *business*, that is, who are its customers, and does it have suppliers and what are its products or services. In this case, it is a Book shop, so its products are books that are for sale. These books are obtained from a range of publishers, for example Penguin books, Pearson, McGraw Hill are example publishers who operate in Australia, and there are many publishers across the world.

Secondly, it is important to understand what the client (in this case the owner who runs the book shop) wants the program/system to be able to do. Let's assume the owner has said the following:

> "I want a computer system that is able to keep track of all the sales I make to customers. I need to know how much they were sold for, and how much it had cost me to originally obtain the book to place it on sale. I need to know how much profit (the amount kept after taking away the cost to obtain the book from the price it was sold for) I make each day, and each week, and each month, as this will help me understand if the business is successful.
>
> Sometimes customers come to the shop to buy a book but find that we have run out, or have not yet obtained a copy of it, so we offer to get it for them. So we want to be able to record all the requests made by customers, which we call customer orders.

© Federation University Australia

> At the end of the year, for tax-purposes, we need to do a 'stock-take' which is to confirm that all the books we have obtained, and haven't sold, are still present in the store. Also, as we have thousands of books, we need a way that we can quickly locate whether a book is in the store and if so where (what shelf) it is on. So we want a way to find information about the books quickly.
>
> We need to know which publisher to contact to order in copies of books, especially popular ones that sell as soon as we get them in. There is specific people at the publishers who we talk to who make sure that we receive the books quickly and then they send us a bill afterwards.
>
> Some customers come back to the shop frequently to get more books, and we want to reward their loyalty by giving them a discount if they spend more than a set amount in a given month – so we want to be able to record the details of these customers. When other customers come wanting books we don't have in stock, who let us order it in for them, we need to record their details as well."

You can probably get a clear sense of what the program needs to do from that narrative. However, there is much that needs to be done to convert that description into working software. An analysis phase will deal with posing and seeking answers to the following questions (amongst others):

1. What information do we need to store?
2. What processes does the software need to be able to perform or support?
3. How should the user expect to interact with the program/system?
4. How should we code a solution?

To answer the first question, the following may be the resultant list of the types of information our program needs to work with, after an initial consideration of the scenario:

- Books: particularly their title, their cost, their selling price, the author, the publisher.
- Customers who place orders: their name, a contact number or email address, the purchases and orders that they make with the bookshop.
- Publishers: their name, their contact person's details (for requesting them to send books), and knowing what set of books they provide.
- Sales: the date, the time, the set of books sold, the payment method type, the amount of money received or entitled to come from credit cards, a transaction ID code (for electronic payments, etc.)

There will probably be other information identified later on.

© Federation University Australia

In answering the second question, perhaps the following is an initial list of processes or tasks that the software needs to provide a means for the user (owner and staff of the bookstore):

- Record that a sale has occurred.
- Record the details of an order being placed by a customer (for items not in the store on the day they came looking)
- Ask for a report about the sales made over the day / week / etc

Further thought may reveal additional tasks, in particular we must always consider that for a program to produce outputs, there must have been corresponding input at an earlier point in time, so these are also tasks the software needs to allow for:

- Add details about new books that will become available, so that the system will have information it can process.
- View details about the books available for sale
- View details about publishers
- Add details about new publishers
- Retrieve orders placed by customers in the past

As we work through implementing these in the program, we might identify further tasks to include. And each of the already identified tasks will require further analysis and solution design until we can write it in working code. Thus solving a problem to produce working computer software is generally an iterative/cyclic process.

In regard to the third question, we need to consider the *form* of interaction by the human with the computer. Traditionally, interaction has occurred through a screen and a keyboard or mouse, or touching the screen (such as on a mobile device). However, more recently many programs are starting to use *voice* as a mode of interaction – for example, Google Maps can tell the user instructions about where and when to turn off one road; other systems may allow the user to give voice commands to the software. Voice is particularly helpful for users who are vision impaired, but also for users who are busy (such as drivers busy with driving on the road). Many programs also use sounds to alert the user to the need for attention (such as alarms, bells or notifications). QR codes and barcodes can also be a mechanism of interaction, where something in the physical world is scanned by the user.

If a screen-based interaction will be used, you need to consider such things as whether it will be using a graphical user interface, or purely textual interface, or a mobile-device

© Federation University Australia

such as phone. For graphical/Mobile based interaction, you then need to consider the design of screens and how the information will be presented and gathered.

The final question identified earlier requires us to think how it will be coded. In this course, the coding will be in Java, but in more general software development, you may need to use a set of languages to write the complete solution, including languages to write native apps, languages to write website back-ends and front-ends, and maybe even need to write code that interacts with external providers (such as a credit card processing authority).

3. Representing an analysis of a program using UML

Sometimes, where the programming language to be used in developing a solution is an object-oriented one (which Java is), we might represent the outcome of our analysis using a range of models from the *Unified Modelling Language* (UML). In particular, a ***class diagram*** is a way of visually representing the set of classes, and their fields and methods, to summarise their designed purposes. You will learn more thoroughly about UML diagrams in ITECH2002.

In class diagrams, the notation we use to represent an individual class, is a box divided into three compartments. The topmost compartment is where we place just the *name* of the class. The middle compartment is where we list the fields/attributes about the objects of the class (what data each object will record). The lower compartment is where we list the methods/operations that the class will provide (the methods which can be called on the individual objects). In Figure 1 we show how we might represent a Book class in the UML Class notation (for a class diagram).

Book
title : String author : String price : double
Book(String,String,double) getTitle() : String getAuthor() : String getPrice() : double setPrice(p : double) toString() : String

Figure 1 - Example of a Class in UML notation

UML has specific syntax by which to indicate the data type of fields, and of parameters and return values. You do not need to show the *names* of parameters (but can for clarity), and can assume the data types of the target language are being used.

© Federation University Australia

As we go through analysing the information and processing needs of a program, we will often find that some pairs of objects seem to have close interaction needs. For example, an order placed by a customer is for one or several books.

When there appears to be a close interaction, we can identify that an **association** exists between the two class types, and this can be represented in a class diagram by a line connecting the two classes. We can often identify a name that describes the relationship (but don't have to).

Most of the time, one of the two objects involved in an association has a degree of responsibility for keeping track of a reference to one or several objects of the other class type. For example, a Sale needs to know about a group of Books which are included in that transaction made by the purchaser at the store. But "Books" generally do not need to know about what Sales they might be included within (though it may be useful to identify such sales later on, but it is not the primary purpose of the program). This concept of one object "knowing about" another, is known as ***navigability***. In the example just given, navigability is from the Sale object(s) to the Book object(s).

We can indicate navigability in a class diagram by using an arrowhead at the end of the association line which is reachable by objects at the other end. So the arrowhead would be at the Book end of the line between Sale and Book.

The most common form of navigability is ***uni-directional association*** which means the association connection can only be identified in one direction (e.g. from a particular Sale to the set of Book objects that form that Sale, but not from any of the Books to any of the Sale objects that might refer to the Book).

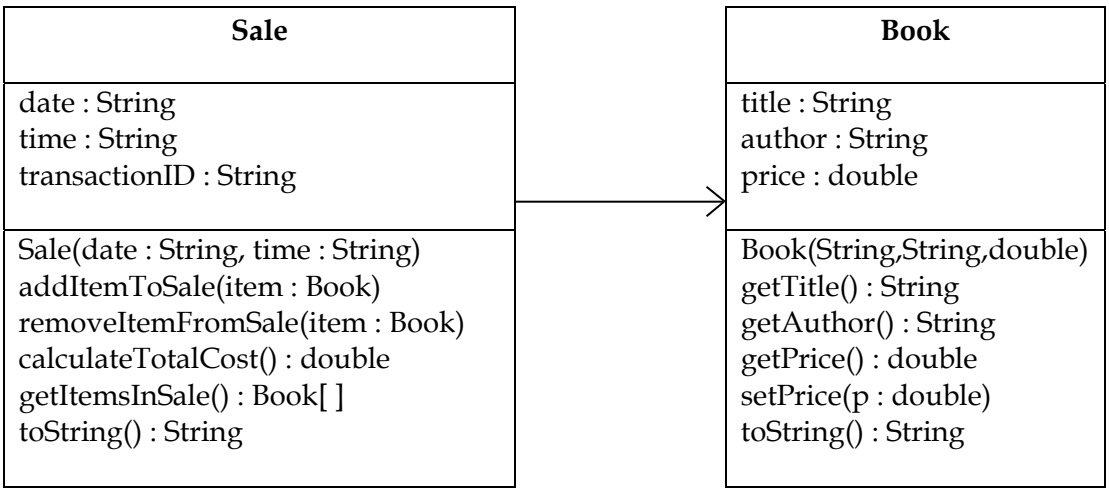


Figure 2 - Example of uni-directional association between two classes

From the representation presented in Figure 2, it is *implied* by the arrow from Sale, that the Sale class will have a field of type Book – this is *how* the association will be manifested in code. We do not explicitly write the field in the middle compartment of the Sale class when the arrow is showing this in the diagram already. You may place the name of the field at the *other class's* end (called a role name, since it explains the role that the other object plays, with respect to the class), i.e. at the same end as the arrowhead.

We can add even further detail to the association line, such as indicating ***multiplicity*** which specifies *how many* of one of the kinds of objects are able to be referred to by the one instance of the other kind. The multiplicity specification is either a asterisk, meaning "any number, possibly 0" or some kind of range, indicated by a lower bound, two dots, then an upper bound (which can be asterisk), attached at the "other" end. So to indicate that a Sale can be for at least 1 book, but at most *any* number of Books (even 1 million), you would add: 1..* above the arrowheads, i.e. at the Book side of the association. The following are some other examples:

- 1 – means exactly and always one.
- 0..1 – means sometimes there is an association (to just one), but at other times there is no association. This is the case for a simple reference variable where the value is sometimes null, and at other times is referring to an actual object.
- 0..* - means sometimes there is no object, but sometimes there is any number of the other object.
- You could say something such as: 2,4,6 which means that at every moment during the program, the first object must refer to either exactly 2, exactly 4, or exactly 6 of the other type of object. Such uses are rare so not important to remember.

4. Association and Aggregation

The most basic type of association, simply means that one object has (as a field, or in a parameter of a method) a reference to another object. This association may be very fleeting/temporary (as in the case of a parameter in a method), or it may be longer lasting (as in the case where a field is referring to the other object).

Sometimes there is a need to represent stronger notions of association. In particular, ***aggregation*** is a type of association where one object *collects a group of* some other object. For example, a Sale is a collection of Books purchased together. A user's feed on a social-media app, is an aggregation of posts made by their friends and favourite

© Federation University Australia

organisations/causes. The posts can simultaneously be included in other users' feeds. The same Books (at least their information) can appear in multiple Sales. Another example is that a University is an aggregation of students, and the students can be aggregated into the courses they are enrolled in – the one Student can simultaneously be included in multiple Courses at the one time.

An even stronger type of association is ***composition*** where the very things being collected form an intrinsic part of the identity of the colleting object. As, a table is composed of the tabletop and the four legs. The legs cannot be part of some other table while they are part of their current table. In a programming example, a screen (window) presented to the user is composed of specific buttons, textboxes and labels which are not simultaneously part of any other screen.

5. Coding Aggregation Relationships in Java

If a particular aggregation identified in our analysis of a program is for a known-maximum limit, we could use an array to code it. We learned about arrays (for primitive types such as int) earlier in the semester. But you can use *any* valid Java data type (as we had even used String) to make an array.

So for example, the following may be the start of the code for the Order class in the bookshop's system:

```
public class Order {
   private Book[ ] orderedItems;          // An array of Book references
   private int numberOrdered;             // The number of elements of array in use.
   private Customer whoPlacedOrder;  // Simple association to one object
   private String date;
   ... ...                                // other fields and methods are required
}
```

The first field, named *orderedItems* is declared as an array of type Book. Since Book will be another domain class, this means that each element of the array that *orderedItems* refers to, will itself be a reference to some Book object. The array itself, will be a set of *references*.

When the constructor of the Order class is done, there will need to be a statement that actually allocates the array elements, for example, maybe there is a standard limit of 10 books per Order:

```
this.orderedItems = new Book[10];          // Allocate a fixed upper limit of 10
```

The individual elements will each be initialised to the value null.

© Federation University Australia

There is then a need to code some mechanism of keeping track of just how many places of the array are currently "in use", as opposed to "free for future use". This is the purpose of the second field shown earlier, *numberOrdered*. It should be initially set to 0. As books are added to the Order, *numberOrdered* should be incremented. If items were to be removed later on, then *numberOrdered* should be decremented (and perhaps the elements of the array be rearranged).

Rather than using the literal value in the line which allocated the array (the 10), it is better practice to use a defined constant. Constants are usually declared prior to the fields, in a class. For example, we could define MAX to be used to represent the maximum number of books that can be ordered at one time:

```
public static final int MAX = 10;
```

Then the array initialisation statement can be updated to be:

```
this.orderedItems = new Book[MAX];          // Allocate the array elements
```

Then, loops and other logic that previously checked on whether the array was full, can instead use MAX (if they weren't already using *length* to check during processing).

5.1. Adding an item into the aggregation

There will normally be some method in the class which is responsible for linking a new item into the aggregation. For the Order class, maybe the 'addOrderedItem' method will accept a Book reference and add it to the array in the next available spot:

```
public void addOrderedItem(Book aBook)
{
    if (numberOrdered < orderedItems.length)        // Check for capacity
    {                                               // If so, then
        orderedItems[numberOrdered] = aBook;        // store the reference
        numberOrdered++;                            // and increment counter
    }
}
```

Here, the if condition uses orderedItems.length to ask the array what its capacity is. It reports how many elements it can hold (some number greater than 0, at least 1); alternatively we could have used MAX to achieve the same. The key is to ensure that the array is not already full.

numberOrdered was initially set to 0 in the constructor. So the first time this method runs, it will still be 0, and so the reference provided in the parameter *aBook* will be copied into position 0 of the array 'orderedItems'. (Thus there is now an alias to the Book).

© Federation University Australia

Then, the *numberOrdered* variable will be increased, to be 1, in readiness for the next time that this method is called (so that the next Book reference goes into element at position 1).

At a subsequent occasion, *numberOrdered* might actually be 10, the value we used to allocate the array, meaning that there have been 10 items already added to the array. We therefore check whether that number (10) is less than the length (in this example, the length is 10). Because 10 is not *less than* itself, the condition will be false, and no more items can be added to the array, so this attempt is not actioned: the reference is not stored, the increment doesn't happen.

Another way to write this method would be to have it return a boolean value indicating either success or failure – you would return true inside this if statement's true-block's body after the increment of numberOrdered. You would add an 'else' block in which to return false. The caller of the method will then be able to examine the returned value to determine whether the attempt to add another Book to the Order's array succeeded or not.

To be thorough, you should also validate that the incoming parameter (*aBook*), is indeed referring to a Book, and not to null. There is no point in adding null into the array or increasing the counter for such a case.

5.2. Retrieving from the aggregate

After having added some items into the aggregate, it is likely that another part of the program would want to retrieve the items from the aggregate.

Since the field which represents the aggregation is private (because it is the aggregate's own data), we need to write a public method to obtain information from the aggregate. We can write the following getItem method:

```
public Book getOrderedItem(int position)
{
    if (position < numberOrdered && position >= 0)  // Is it a valid used position?
    {                                               // If so, then
       return orderedItems[position];               // return the specific Book
    }
    else  return null;                              // We must return something
}
```

Here, the if condition checks that the parameter *position* is one that represents an actually-used spot in our array, and if so it then returns the Book object reference that is found in that place of the array. Otherwise, it returns *null* to signify that the position was not a valid value. (Since we must always return something from this method, null is our default value when an invalid position is given).

© Federation University Australia

In different situations, you may need to obtain an item from an aggregate using some piece of information – for example, maybe we need the Book but only know its title (or its author, or its barcode). We can write other methods that will accept a different parameter, and utilise the parameter to do a comparison of the relevant field of each item in the aggregate.

5.3. Never return the array representing the aggregation

You may be tempted to write a method that returns the array that represents the aggregation of the Books. Although Java lets you write such, it is strongly suggested that you should not do this, because it will enable other code to bypass the encapsulation and responsibilities which belong properly to the Order class (the aggregate) and so is considered to be poor programming style. Allowing the privately stored reference to the aggregate to be returned would constitute what we term a ***privacy leak***.

If there is a genuine need for returning an array (or other collection/group) that contains all the objects that were added to the aggregate, the better technique is as follows: Create a *fresh new* array, and copy the references of the aggregate into the new array, and return that new array. This way, it will not matter what is done to this duplicate array, as it will have no affect to tamper with the one that manages the aggregate.

6. ArrayList class

Some people find it cumbersome to use arrays to implement aggregation, and it is prone to bugs because of needing to ensure a separate *counter* is kept up to date. Additionally, arrays have a fixed size limit and if resizing is to be supported, the process of creating a replacement array that is larger than the original, and then copying the items from the original to the new array, can be annoying.

Fortunately Java provides a range of pre-written classes that can help us to manage collections of objects in a more dynamic manner. One of these is the ArrayList class.

An ArrayList is an *object* which has various methods we can call on the object in order to add elements, remove elements, retrieve elements, find out how many elements it currently has, and various other actions.

The ArrayList class requires the use of a special new syntax both when declaring a variable (or return type or parameter), and when instantiating (creating) the object that is assigned to the variable. An example is the following:

```
ArrayList<String> seasons = new ArrayList<String>();
```


© Federation University Australia

The angular brackets must come straight after the class name ArrayList. Inside the brackets we must place *any class type* that the compiler would understand. In the above case, we used the String class. But we could just as easily use the Book class or the Sale class from the earlier examples, e.g.:

```
ArrayList<Book> allBooksInStore = new ArrayList<Book>();
```

Note also, that we must still include the round brackets when using 'new', after the close angled-bracket.

The angular brackets mean that the ArrayList is a *parameterised class* – there is a basic definition of ArrayList, but the definition includes some parameters. These are different to parameters that we are used to providing when calling methods. The parameters in this case are parameters to help define a specific class. It means that the ArrayList is a type of template, but we can make particular versions by specifying the required parameters. In this case, there is a need for just one parameter, which must be a class-type (such as String, Book, Sale). The *name* given to the class parameter in the definition of the ArrayList class, is E – meaning *element type*. You will notice this E appearing in the method declarations in the ArrayList class, as a placeholder for whatever class we specify when we declare an ArrayList variable.

Activity Look at the API documentation page for ArrayList to find where the <E> appears in the documentation.
(Hint: look at method signatures)

What can we do with an ArrayList? The following table shows a summary of key methods provided by the ArrayList class, for the seasons ArrayList (whose elements are String objects):

Method	Meaning	Example
add(E item)	Add the item specified in the parameter, to the end of the list of items stored in this ArrayList	seasons.add("Winter");

add(int index, E item)	Insert the item specified in the second parameter, at the position indicated by the first parameter. Shifts all subsequent elements to be one index later.	seasons.add(0,"Autumn");
		// adds Autumn to front (0),
		// shifts Winter to be at 1.

E get(int index)	Retrieve the item from the position as specified by the parameter	String aSeason = seasons.get(0);
E remove(int index)	Removes the item from the position specified by the parameter, returning it to the caller. Shifts subsequent items back by one position.	seasons.remove(0);

boolean remove(Object o)	Removes the first occurrence of the specified object (given as parameter) from the ArrayList, if it is present	String s = "Spring";
		seasons.add(s); // adds object
		seasons.remove(s); // removes it

int size()	Returns a number telling how many items are currently in the ArrayList	int total = seasons.size();
boolean isEmpty()	Returns true if the size is 0, otherwise returns false	if (seasons.isEmpty()) System.out.println("No seasons");

6.1. Coding Aggregation using ArrayList in Java

The earlier example can make use of ArrayList. The following may be the start of the code for the Order class in the bookshop's system:

```
public class Order {
    private ArrayList<Book> orderedItems;        // An ArrayList of Book references
    private Customer whoPlacedOrder;             // Simple association to one object
    private String date;
    ... ...                                      // other fields and methods are required
}
```


© Federation University Australia

The method to add a Book to the order can be rewritten this way:

```
public void addOrderedItem(Book aBook)
{
    if (aBook != null) {                    // No point adding nothing.
       orderedItems.add(aBook);             // store the reference
    }
}
```

The method to retrieve from a position within the aggregate becomes:

```
public Book getOrderedItem(int position)
{
    // Is it a valid used position?
    if (position < orderedItems.size() && position >= 0)
    {                                                   // If so, then
       return orderedItems.get(position);                   // return the specific Book
    }
    else  return null;                                  // We must return something
}
```

As with arrays, we should never directly return the ArrayList object tat is used inside the aggregator object (the Order class in this case). Doing so would mean that the caller would receive the reference to the ArrayList object, which is meant to be kept secret inside the Order object. The caller could then use that reference to manipulate the contents of the ArrayList without any oversight by the Order object (potentially making malicious or unapproved changes).

If you need to return an ArrayList containing the items that constitute the aggregate, you should instead make a *clone* of the ArrayList. A clone is a new separate object – in this case, a freshly made ArrayList which will have *the same elements*, but being a separately made ArrayList object it is at a separate memory location. Thus any subsequent changes made to the new ArrayList will not impact the original ArrayList in any way. The following is an example of code to return a clone:

```
public ArrayList<Book> getOrderedItems()
{
    ArrayList<Book> duplicate;
    duplicate = (ArrayList<Book>) orderedItems.clone();
    return duplicate;
}
```

We need to include a typecast in front of the call of clone(), because clone is declared to return something slightly unexpected.

© Federation University Australia

7. Data Management Classes

In many cases, we need to have some centrally-located object that can provide a convenient place to store a set of objects, and act somewhat like a database. For example, the bookshop has a large set of Books that could potentially be included in orders and sales. We want to have an object that can take the responsibility of being the Book Database.

A ***data management class*** is a class which defines a set of public methods that provide a means to add, retrieve or search for items that meet some criteria, for a specific type of object. For instance, we might make **BookDatabase** to be a class in which every Book object is stored, we might make **SaleDatabase** to be a class in which every Sale object is stored.

Internally, the data management class will likely use an ArrayList (or some alternatives which we have not yet learnt about) to aggregate all the individual items that are added into the class. Or at a later time, the behaviour of the methods could be updated to instead store and retrieve from an external database or cloud-hosted data storage solution. The advantage is that the other classes in the system which *use* the data management class will not need to be concerned with exactly *how* the storing is being achieved; it needs only be concerned about how to use the methods.

As an Example, the BookDatabase class could provide the following methods:

- addBook(Book someBook) – to add new book objects, so that it could be retrieved later. It should probably prevent a particular instance from being added twice;
- retrieveBookByISBN(String ISBN) – to return the Book object whose ISBN field's value matches the value provided in the parameter;
- retrieveBooksByAuthor(String author) – to return a group of Book objects where the author field's value matches the value provided in the parameter;
- getListOfAllBooks() – could return an ArrayList, or an array, of all the Book objects. Care would need to be taken not to cause a privacy leak of the internal aggregation field (e.g. by returning a clone instead).

The program would need to create an instance of the BookDatabase sometime early in the setting-up phase. It is likely to be needed within the class which either manages a menu or the individual use-cases (if they are in separate classes), since most use cases will need convenient access to the data.

© Federation University Australia

8. Summary

In this week's module, we commenced by considering some of the issues we need to address to generally solve problems that will require a computer program to be written. We can represent the outcome of the planned design using UML class diagrams, which show the set of classes, their fields and methods, and also indicate the associations (relationships) that objects of different class types may have with each other during run-time.

We then examined some of the techniques we can implement aggregation by using arrays in conjunction with a variable that keeps track of the number of items currently "in" the aggregation.

We then introduced the ArrayList class, which is a class that requires a parameter to be specified when making variables and objects using the < > notation. Any class type, such as String or even our own types such as Book, Orders, and Students, can be used for the type parameter, to indicate what type of objects are going to be stored in the ArrayList.

We saw the importance of avoiding *privacy leaks* which would arise if the internal field of a class, where that field is an object reference to a mutable object (such as an ArrayList, or even an array) was directly returned to an external caller. To prevent a privacy leak requires returning either a clone or brand-new ArrayList, so that the original cannot be altered by other parts of the program.

Finally we considered the concept of a data management class, as one which provides a simple interface (set of methods) by which to store and retrieve objects in a manner similar to that afforded by a usual database.

Read Parsons (2nd ed.), Chapter 7 up to end of Section 7.4, pp. 141 – 165.
You may read the remainder of the chapter if you wish.

Key Terms

client
requirement
analysis
design
UML
class diagram
association
uni-directional association
multiplicity
role-name
aggregation
privacy leak
data management class

The following page has been left blank

© Federation University Australia

© Federation University Australia